Hybrid MPI-CUDA Framework for High-Performance Large-Scale Log Analysis

PROJECT DOCUMENTATION

MPI • CUDA • Hybrid Parallel Log Processing

Ubuntu 26.04 LTS | NVIDIA RTX 3050 4 GB | CUDA 13.4 | 12 CPU cores

1. Project Overview

This project implements Serial CPU, CUDA-only GPU, MPI-only CPU parallelism, and Hybrid MPI-CUDA approaches for large-scale log analysis. It counts ERROR, WARN, INFO, and DEBUG levels and compares execution time.

2. Problem Statement

The project investigates whether CPU process parallelism, GPU acceleration, or their combination provides the best performance for large log files.

3. Objectives

• Generate synthetic datasets.

• Implement four processing variants.

• Verify matching results.

• Benchmark approximately 100 MB to 2 GB.

• Analyze speedup, efficiency, and the best approach.

4. System Requirements

Component	Configuration
OS	Ubuntu 26.04 LTS
CPU	12 cores
GPU	NVIDIA RTX 3050, 4 GB
CUDA	13.4
RAM	~7.6 GB
MPI	mpirun-capable MPI runtime
Languages	C, CUDA C++, Python, Bash

5. Project Structure

• Makefile

• generate_logs.py

• benchmark.sh

• analyze_results.py

• src/serial_main.c

• src/mpi_only_main.c

• src/cuda_only_main.c

• src/main.c (Hybrid)

• src/cpu_analyze.c

• src/kernel.cu

• include/cpu_analyze.h

• include/kernel.h

6. Dataset and Log Format

Example: 2026-09-12T... [ERROR] User login successful. Generator weights are approximately ERROR 5%, WARN 15%, INFO 60%, DEBUG 20%.

7. Implementation

7.1 Serial CPU

Single CPU path scans the file and counts the four levels.

7.2 CUDA-only

Transfers data to GPU, launches a CUDA kernel, scans in parallel, and copies counters back. The corrected kernel searches for '[' after line starts.

7.3 MPI-only

Splits the file into byte ranges, aligns boundaries to newlines, analyzes chunks on CPU ranks, and combines counts with MPI_Reduce.

7.4 Hybrid MPI-CUDA

MPI partitions the file; each rank processes its chunk on the GPU and contributes results to MPI_Reduce.

8. Correctness Fix

The original matching assumed the keyword was at the beginning of a line, but timestamps precede it. The CPU and CUDA logic was corrected to find '[' after line starts, compare the keyword, and require ']'.

9. Correctness Verification

Metric	Result
Dataset	92,152,406 bytes (~92 MB)
Lines	1,500,000
ERROR	75,056
WARN	224,933
INFO	900,082
DEBUG	299,929
MPI	2 and 4 processes matched
Hybrid	2 and 4 processes matched

10. Build and Run

```
make
python3 generate_logs.py 1500000 logs_100MB.log
./serial_analyzer logs_100MB.log
mpirun --oversubscribe -np 4 ./mpi_only_analyzer logs_100MB.log
./cuda_only_analyzer logs_100MB.log
mpirun --oversubscribe -np 4 ./hybrid_analyzer logs_100MB.log
```

11. Benchmark Procedure

Benchmarks covered Serial, CUDA-only, MPI-only at 1/2/4/8 processes, and Hybrid at 1/2/4/8 processes for approximately 100 MB, 500 MB, 1 GB, and 2 GB datasets. The 2 GB dataset used 3,400,000 lines.

12. Experimental Results

Dataset	Serial	CUDA	MPI best	Hybrid best	Best
100 MB	0.1343	2.3638	0.1367 (4)	1.0894 (2)	Serial
500 MB	0.6575	1.9245	0.1277 (8)	1.0853 (2)	MPI
1 GB	1.4727	2.3270	0.2424 (8)	1.4095 (4)	MPI
2 GB	0.2725	2.8062	0.0560 (8)	0.6759 (8)	MPI*

*The 2 GB serial time is unusually low relative to 1 GB and should not be used to claim smooth scaling.

13. Why MPI Was Faster

The workload is simple text processing: locating a level and incrementing a counter. MPI uses multiple CPU processes without CPU-GPU transfers. CUDA adds transfer, kernel-launch, synchronization, and result-transfer overhead. Hybrid adds those GPU costs to MPI overhead and shares one RTX 3050 among ranks.

14. When Hybrid Could Become Faster

• More computation per byte.

• Complex parsing/pattern matching/hashing/anomaly detection.

• Larger GPU batches.

• Optimized CUDA memory access and fewer atomics.

• Stronger GPU hardware.

• Reduced or overlapped CPU-GPU data movement.

15. Limitations

• Single 4 GB GPU.

• Simple workload.

• Timing affected by cache/system load.

• 2 GB serial result is anomalous.

• Efficiency above 1 in some MPI rows is a baseline/timing artifact.

16. Conclusion

MPI-only was the fastest measured approach for larger datasets because the workload is CPU-parallelizable and GPU overhead dominates. Hybrid remains a valid architecture and may win for more computationally intensive workloads. Approximately 2 GB is sufficient as a final experimental dataset for this project.

17. Viva Summary

What is the project? A hybrid MPI-CUDA framework for parallel large-scale log analysis.

Why MPI is fastest? Simple CPU-parallel workload with no GPU transfer overhead.

Why CUDA is slower? GPU overhead is large relative to the computation.

Why Hybrid is slower? It combines MPI and GPU overhead while sharing one GPU.

Can Hybrid become faster? Yes, for more computationally intensive GPU-friendly workloads.